Sharding Database Partitioning Explained


explain this in details

The slide explains **Sharding**, a technique for partitioning data across multiple databases to improve scalability and performance. Below is a detailed explanation of each point mentioned:

What is Sharding?

Sharding is a database architecture pattern where data is split into smaller, more manageable parts called **shards**. Each shard is stored on a separate database instance. This technique helps in distributing the load, improving performance, and making large databases more scalable.

Partitioning in Sharding

Sharding is typically done based on **partitioning attributes**, also known as **partitioning keys** or **shard keys**. A common example is using **User ID** to distribute data across multiple databases.

Example:

- Imagine a user database where each user is assigned a unique ID.
- We can partition the data such that:
  - User records with **ID 1 to 100,000** are stored in **Database 1**.
  - User records with **ID 100,001 to 200,000** are stored in **Database 2**, and so on.
- This way, queries related to a specific user can be routed to the corresponding database, reducing the load on a single database server.

How Sharding Works in Applications

Since data is distributed across multiple databases, the application must:

1. **Track where each record is stored** – The application should know which database contains a specific piece of data.
2. **Route queries correctly** – When fetching, updating, or deleting data, the application must direct queries to the right database.

Advantages of Sharding

✅ **Scalability**: Sharding allows a system to handle large amounts of data efficiently by spreading the load across multiple databases.

✅ **Performance**: Queries can run faster as each database manages a smaller portion of the total data.
✅ **Easy to implement**: Basic sharding strategies (like range-based partitioning) are straightforward to implement.

Drawbacks of Sharding

❌ **Lack of Transparency**:

- The application needs to handle query routing, which increases complexity.
- Queries spanning multiple shards (e.g., aggregations, joins) are difficult to execute efficiently.

❌ **Difficult to Rebalance Load**:

- If one database becomes overloaded, redistributing the data (re-sharding) is complex.
- Moving records between shards can disrupt services.

❌ **Increased Risk of Failure**:

- More databases mean more points of failure.
- If one shard fails, a part of the data may become unavailable.
- To prevent failures, **replicas** (copies of data) must be maintained, increasing the maintenance overhead.

Conclusion

Sharding is an effective strategy for scaling databases, but it introduces challenges such as increased complexity in query management, difficulty in rebalancing the load, and higher failure risks. It is commonly used in large-scale applications like social media platforms, e-commerce websites, and cloud-based services.


explain in details

The slides discuss **Key-Value Storage Systems**, a type of NoSQL database that stores data as key-value pairs. Let's break down the concepts in detail.

What are Key-Value Storage Systems?

Key-value storage systems are a type of database that stores data in the form of key-value pairs. These databases are highly scalable and are designed to handle **large numbers of records (billions or more)**, where each record is typically small in size (in KB or MB).

Characteristics of Key-Value Stores

1. **Partitioned Across Multiple Machines**
    - The data is distributed across several machines to balance the load and improve performance.
    - This ensures that the system can scale horizontally, meaning more machines can be added as data grows.
2. **Query Routing**
    - When a query is made, the system determines which machine contains the requested key and routes the query there.
    - This avoids unnecessary data scanning and ensures efficient retrieval.
3. **Replication for Availability**
    - The same data is stored on multiple machines to prevent data loss in case of failure.
    - **Consistency Mechanisms**: Ensuring updates are applied to all replicas to keep the data **consistent**.

Examples of Key-Value Stores

- **Amazon S3, Amazon DynamoDB** – Examples of distributed key-value stores used in cloud computing.

Types of Data Stored in Key-Value Stores

Key-value stores can store different types of data structures, including:

1. **Uninterpreted Bytes (Raw Data)**
    - Stores raw binary data with an associated key.
    - Used in storage services like **Amazon S3, Amazon Dynamo**.
2. **Wide-Table Format**
    - Each key is associated with multiple attributes instead of a single value.
    - Similar to columnar databases, enabling flexible schema designs.
    - Used in:
        - **Google BigTable**
        - **Apache Cassandra**
        - **Apache HBase**
        - **Amazon DynamoDB**
    - Advantage: Some operations like filtering can be executed directly on the storage node, improving query efficiency.
3. **JSON (JavaScript Object Notation)**
    - Stores data in a flexible document format.
    - Examples:

        - **MongoDB**
        - **CouchDB**
    - This format is widely used in **document-oriented databases**.
4. **Document Stores**
    - Special type of key-value stores that handle **semi-structured data** (like JSON).
    - Supports nested objects and complex queries.
5. **Versioned Data**
    - Some key-value stores maintain **multiple versions of the same data**.
    - They use **timestamps** or **version numbers** to keep track of updates.

Conclusion

Key-value storage systems are highly efficient for handling large-scale data with fast lookups. They are widely used in distributed computing, cloud services, and big data applications. Depending on the use case, they can store simple key-value pairs, structured tables, JSON documents, or even versioned data.

The slide explains the **MapReduce Paradigm**, a programming model designed for processing large-scale data in a distributed and parallel computing environment. Below is a detailed breakdown:

What is MapReduce?

MapReduce is a computational framework that enables **parallel processing of large datasets** across multiple machines in a distributed system. It provides a **reliable and scalable platform** for handling massive amounts of data efficiently.

Key Features and Benefits

1. **Scalable and Reliable Parallel Computing**
    - Designed to run on clusters of machines, making it **scalable** as more machines can be added.
    - Ensures **fault tolerance**, meaning if one machine fails, computations can be recovered.
2. **Abstraction of Distributed Computing**
    - Developers do not need to handle **low-level parallelization** or **distributed system complexities**.
    - The framework abstracts these issues, allowing programmers to focus on writing logic.
3. **Two Key Functions in MapReduce**

- **Map Function (`map()`)**
  - Processes input data and produces key-value pairs.
  - Example: Counting words in a document → `map()` would output (word, 1) for each occurrence.
- **Reduce Function (`reduce()`)**
  - Aggregates or processes key-value pairs from the `map()` step.
  - Example: Summing up all occurrences of each word.

4. **Automatic Parallelization**
   - The system takes care of **dividing tasks**, **distributing them across machines**, and **coordinating execution**.

Historical Background and Examples

- The concept of **MapReduce** has existed for decades.
- **Google MapReduce**: Introduced by Google to process web-scale data.
- **Hadoop**: An open-source implementation of MapReduce, widely used for big data processing.

Data Storage in MapReduce

- Typically works with **distributed file systems** like:
  - **HDFS (Hadoop Distributed File System)** – Commonly used with Hadoop.
  - **Key-Value Stores** – Used to store intermediate or final results.

Conclusion

MapReduce is a powerful framework for processing **big data** across distributed clusters efficiently. By abstracting the complexities of parallelization, it allows developers to focus on logic rather than infrastructure, making it a core component of many **big data** and **machine learning** applications.

> I understand tyhe map part but what does the reduce do?

Great question! While the **map()** function processes data and emits key-value pairs, the **reduce()** function is responsible for aggregating and processing those key-value pairs to produce the final result.

How Reduce Works

1. **Grouping by Key**

- The output from the `map()` function is **shuffled and sorted** so that all values for a particular key are grouped together.

2. **Processing the Groups**
   - The `reduce()` function then **combines, aggregates, or processes** the values associated with each key.

Example: Word Count

Let's break it down step by step:

Step 1: Map Function Output

If we process a document using `map()`, it might output key-value pairs like:

```
arduino

`("word1", 1)
("word2", 1)
("word1", 1)
("word3", 1)
("word2", 1)
`
```

Step 2: Shuffle & Sort

The system groups the values by key:

```
arduino

`"word1" → [1, 1]
"word2" → [1, 1]
"word3" → [1]
`
```

Step 3: Reduce Function

Now, the `reduce()` function will sum up the values for each word:

```
arduino

`"word1" → 2
"word2" → 2
"word3" → 1
`
```

So the final output of `reduce()` is:

```
arduino

`("word1", 2)
("word2", 2)
("word3", 1)
`
```

Other Examples of Reduce Function

1. **Finding Maximum Temperature in Weather Data**
   - `map()` outputs (year, temperature) pairs.
   - `reduce()` finds the **maximum temperature** for each year.
2. **Counting Log Entries by Type**
   - `map()` outputs (error type, 1) for each log entry.
   - `reduce()` sums up the occurrences of each error type.

In Summary

- `map()` processes input data and emits key-value pairs.
- `reduce()` **aggregates, processes, and summarizes** the mapped data.
- Without `reduce()`, the data would remain unprocessed and unorganized.

Would you like another example with code? 😊


explain this

This diagram illustrates the **Parallel Processing of a MapReduce Job**, showing how data flows through the different stages of a MapReduce system. Let's break it down step by step:

---

1. User Program and Master Node

- The **User Program** submits a job to the **Master** node.
- The **Master** node is responsible for:
  - Assigning **map** tasks to available worker nodes.
  - Assigning **reduce** tasks after the map phase is complete.

---

2. Input File Partitions

- The input dataset is split into **multiple partitions** (`Part 1`, `Part 2`, ..., `Part n`).
- Each partition is read and processed by a **Map Task** (`Map 1`, `Map 2`, ..., `Map n`).

---

3. Map Phase

- Each **Map Task** takes a partition of data and processes it to generate key-value pairs.

- These intermediate key-value pairs are written **locally**.

---

4. Shuffle & Sort Phase (Intermediate Files)

- The intermediate key-value pairs are:
  - **Grouped by key** (so all values for a given key are together).
  - **Transferred across nodes** (remote read).
  - **Sorted** before they are processed by the reducers.

---

5. Reduce Phase

- The **Master** assigns **Reduce Tasks** (`Reduce 1`, `Reduce 2`, ..., `Reduce m`).
- Each reducer takes a group of key-value pairs and processes them (e.g., summing values, filtering data, aggregating results).
- The processed data is written to **output files**.

---

6. Output Files

- The final reduced output is stored in **multiple files** (`File 1`, `File 2`, ..., `File m`).
- These files contain the processed results, ready for further use.

Key Takeaways

- **Parallel Execution:** Multiple mappers and reducers run in parallel to process large datasets efficiently.
- **Distributed Processing:** Data is distributed across multiple machines.
- **Fault Tolerance:** If a node fails, tasks can be reassigned by the master.
- **Scalability:** The framework can handle large-scale data processing.

Would you like a real-world example of how this works? 😊